

Mock Design: A Ticket Booking System

Phase 4 · Interview-Ready · Estimated time: 2 hours

1. Concept From First Principles: Applying the Framework

Ticket booking (BookMyShow/District/IRCTC-style) pushes Chapter 10's inventory-correctness problem to its most extreme, most concentrated form: instead of thousands of units of stock spread across many products, you have a small, fixed number of specific, individually-identified seats (a single cricket final might have 40,000 total seats), and an enormous simultaneous crowd competing for exactly the same small inventory the instant it goes on sale. This concentration — not raw scale — is what makes this problem distinct and genuinely harder than Chapter 10's general checkout.

Step 1 — Clarify requirements

- Functional: browse events/shows and available seats; select specific seats; complete payment to confirm booking; view a live, accurate seat map.
- Non-functional: a seat must never be sold to two people; the system must survive a massive simultaneous traffic spike the instant a popular sale opens (a well-known, extreme real-world load pattern for this domain), without either overselling or becoming unresponsive.

Step 2 — Estimate scale

Assume a high-demand event with 40,000 seats and 2 million users attempting to book within the first minute of sale opening — a 50:1 demand-to-supply ratio, concentrated into a single, extremely narrow time window. This is a fundamentally different scale problem than steady-state throughput: the design must survive a massive instantaneous spike against a small, fixed, non-renewable pool of inventory, not a sustained high request rate against effectively unlimited stock.

Step 3 — API

POST /seats/{seatId}/hold {userId} -> {holdId, expiresAt}, POST /bookings {holdId, paymentMethodId} -> {bookingId, status}. Splitting seat selection into a distinct **hold** step, separate from final booking, is itself a key design decision worth stating explicitly at the API level: it lets a user complete payment without racing every other user for the same seat during the payment flow itself.

Step 4 — High-level design

Client → Load Balancer → Booking Service, backed by a Seat Inventory Store tracking each individual seat's state (AVAILABLE -> HELD -> BOOKED). Given the extreme instantaneous spike at sale-open, a **waiting-room / queueing layer** in front of the Booking Service is standard practice at this scale — admitting users into the actual booking flow at a controlled rate rather than letting 2 million simultaneous requests hit the seat-selection system directly, which would fail regardless of how well the backend itself is architected.

Step 5 — Deep dive: seat locking and preventing double-booking under extreme contention

Seat locking mechanism: when a user selects a seat, it transitions from AVAILABLE to HELD using an atomic conditional update (the same pattern as Chapter 10's inventory decrement: `UPDATE seats SET status='HELD', held_by=X WHERE seat_id=Y AND status='AVAILABLE'`), with a short TTL (typically 7–10 minutes, implemented via Redis with automatic key expiration, chosen specifically because Redis's TTL mechanism handles the automatic release-back-to-available step without a separate cleanup job). If the user doesn't complete payment within the hold window, the seat automatically reverts to AVAILABLE for others.

Why the waiting room matters architecturally, not just as a UX nicety: without it, 2 million concurrent seat-selection attempts would all hit the atomic-update layer simultaneously; even though each individual atomic update is correct, the database is now serializing an enormous burst of contention on a small set of rows (only 40,000 seats exist, so contention concentrates heavily), which can degrade latency for everyone or exhaust connection pools. A waiting room admits users at a rate the booking backend can actually sustain, converting an uncontrolled stampede into a controlled, bounded flow — directly reusing this program's rate-limiting instincts from Chapter 4, applied here to protect a downstream system rather than an API itself.

A genuinely tricky follow-up: what if a user holds a seat, then their payment fails? The hold must be explicitly released back to AVAILABLE immediately (not just left to expire via TTL, which would needlessly keep the seat locked for the remaining hold window) — a compensating action, in the same Saga-style spirit as Chapter 10's checkout, triggered directly by payment failure rather than relying solely on the passive TTL expiry as a fallback safety net.

Step 6 — Trade-offs and wrap-up

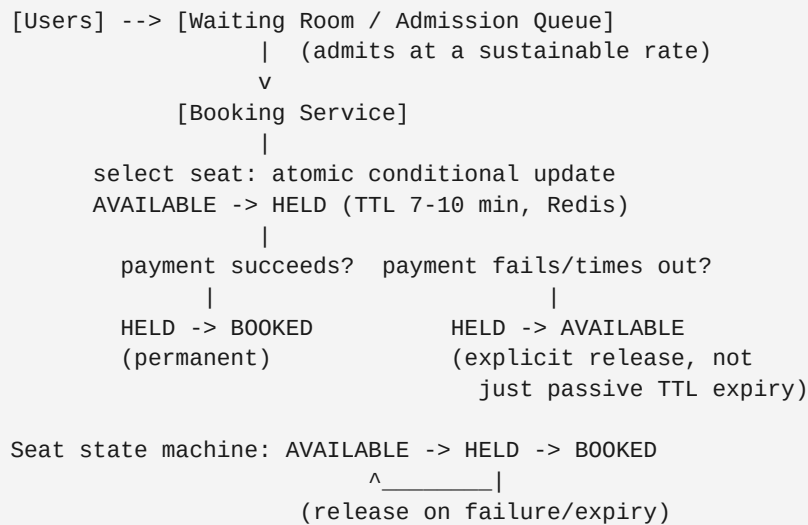
Volunteer: a fixed 7–10 minute hold window is itself a trade-off — too short frustrates genuine users who need time to review their selection and pay; too long needlessly locks scarce seats away from other users during exactly the highest-demand window. Name this as a tunable parameter worth monitoring (Phase 3, Ch 13) and adjusting based on observed real payment-completion-time data, not a fixed constant chosen once and forgotten.

2. Real-World Example / Case Study

BookMyShow's well-documented seat-locking approach and IRCTC's famously extreme Tatkal booking rush (a daily event where a fixed number of train seats open for booking at an exact second and are typically gone within moments) are the canonical, widely-discussed real-world instances of exactly this concentrated demand-spike problem — IRCTC's own publicly reported historical struggles with this exact load pattern are a useful, concrete illustration of why the waiting-room and atomic-locking patterns in this chapter aren't theoretical conveniences but genuine production necessities.

3. Diagram

Sale opens: 2,000,000 users attempt to book 40,000 seats



4. Trade-offs Table

Decision	Alternative	Why this choice
Waiting room / admission queue at sale-open	Let all requests hit the booking backend directly	Converts an uncontrolled stampede into a bounded, sustainable flow
Separate hold step before payment	Lock seat only at final payment confirmation	Lets users complete payment without racing others for the same seat
Redis TTL for automatic hold expiry	Manual cleanup job scanning for expired holds	Automatic expiration avoids needing a separate polling/cleanup process
Explicit release on payment failure	Rely solely on TTL expiry	Frees the seat immediately instead of needlessly holding it for the full window

5. Common Interview Questions

Q1: "How is this different from the general checkout system in Chapter 10?"

Model approach: Name the concentration difference explicitly — a small, fixed, individually-identified inventory pool facing an enormous simultaneous demand spike at one exact instant, versus checkout's more generally distributed inventory and traffic pattern.

Q2: "Why not just lock the seat at the moment of final payment, skipping the earlier hold step?"

Model approach: Explain that without an earlier hold, a user could complete an entire payment flow only to discover the seat was taken by someone else in the meantime — a poor user experience the hold step exists specifically to prevent.

Q3: "How would you handle a scenario where the sale-open moment causes far more load than your waiting room can admit at a sustainable rate?"

Model approach: Discuss the waiting room's own scaling (horizontal scaling of the admission-queue layer itself, Phase 1 Ch 6) as a separate, independent concern from the booking backend's scaling, and note that a fair, randomized or first-come-first-served admission order matters for perceived fairness even under extreme load.

6. Practice Exercises

- 1 Run the full six-step framework for "design a ticket booking system like BookMyShow," out loud, with a timer.
- 2 Write the exact atomic conditional update for the AVAILABLE-to-HELD seat transition, and explain what happens when two users attempt it simultaneously for the same seat.
- 3 Design the waiting-room admission mechanism in more detail: what determines admission rate, and how do you communicate wait status to users still queued?

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"System Design of a Ticket Booking System: BookMyShow"	Directly covers seat locking and inventory concentration for this exact problem.
English	"IRCTC System Design Railways Ticket Booking Service System Design Interview Question"	Covers the extreme Tatkal-style demand spike scenario referenced in this chapter.
Hindi	"System Design 2: Design Ticket Booking System like BookMyShow, District HLD LLD"	Hindi/Hinglish walkthrough of seat booking and locking mechanisms.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Bilingual course covering concurrency and locking patterns as part of the broader curriculum.

8. Key Takeaways

- Concentrated demand against a small, fixed inventory pool at one exact instant is what makes this problem distinct from Chapter 10's general checkout.
- A waiting room / admission queue protects the booking backend from an uncontrolled stampede, independent of how correct the locking logic itself is.
- A separate hold step (before payment) prevents users from racing for a seat during the payment flow itself.
- Redis TTL-based automatic expiry handles the common case; explicit release on payment failure handles the fast-failure case without waiting out the full TTL.
- Atomic conditional updates remain the core correctness mechanism, identical in principle to Chapter 10's inventory decrement.
- The hold-window duration is a genuine, tunable trade-off between user experience and inventory availability, not a fixed constant.

- IRCTC's real Tatkal booking rush is a widely known, concrete illustration of why these patterns are production necessities, not theoretical exercises.